

Homework 1

Section 0

Field	Value
SID 4	1346
PORT_BASE	8446
PREFIX	s1346
SEED	1346
VERIFY_SEED	261346
DOMAIN_ID	2 - Municipal Transit Incidents
Hardware	MacBook Pro 13”(M2, 8GB RAM)
Local Model	qwen2.5:3b (substituted for qwen3:8b due to 8GB RAM constraint)
Github Repository	github.com/Himja18/data260-1346
Tagged commit (HW1)	d1c550948d7ebaa9468baf36a9688ae52c3ec cbd
AWS ECS endpoint	18.224.137.47:8446

Part 1: Web Application using HTML and Javascript

A transit-signage-themed incident report form was built for Municipal Transit Incidents (Domain 2). The design uses a route-line color strip (matching the four incident categories: Delay, Breakdown, Accident, Service Change), a monospace route-number field, and Archivo/IBM Plex typography to evoke a digital transit departure board.

HTML Code:
index.html

```
index.html x agents_demo.py model_client.py hw1_client.py METRICS.md AI_USE.md M IF_RUN_LOG.txt verification.json nondeterminism_input.json run ...
code > web_application > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>HW1-Hinja Patel</title>
7 <link rel="preconnect" href="https://fonts.googleapis.com">
8 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
9 <link href="https://fonts.googleapis.com/css2?family=Archivo:wght@700;800&family=IBM+Plex+Mono:wght@500;600&family=IBM+Plex+Sans:wght@400;500;600&display=swap" rel="stylesheet">
10 <style>
11 :root {
12   --ink: #16213E;
13   --amber: #F2A93C;
14   --paper: #F7F5EF;
15   --steel: #5B6472;
16   --line: #D8D3C4;
17   --card: #FFFFFF;
18   --delay: #F2A93C;
19   --breakdown: #5B6472;
20   --accident: #C4483C;
21   --service-change: #2E6E9E;
22 }
23
24 * { box-sizing: border-box; }
25
26 body {
27   font-family: 'IBM Plex Sans', Arial, sans-serif;
28   margin: 0;
29   padding: 48px 20px 80px;
30   background: var(--paper);
31   background-image:
32     radial-gradient(circle at 100% 0%, #242169 0%, #242169 0.08% 0%, transparent 45%),
33     radial-gradient(circle at 0% 100%, #242169 0%, #242169 0.05% 0%, transparent 45%);
34   color: var(--ink);
35   min-height: 100vh;
36 }
37
38 .page {
39   max-width: 620px;
40   margin: 0 auto;
41 }
42
43 .masthead {
44   display: flex;
45   align-items: center;
46   gap: 12px;
47   margin-bottom: 4px;
48 }
49
50 .line-dot {
51   width: 14px;
52   height: 14px;
53   border-radius: 50%;
54   background: var(--amber);
55   box-shadow: 0 0 4px #242169 0.25;
56   flex-shrink: 0;
57 }
58
59 .eyebrow {
60   font-family: 'IBM Plex Mono', monospace;
61   font-size: 0.72rem;
62   letter-spacing: 0.14em;
63   text-transform: uppercase;
64   color: var(--steel);
```

```

index.html X agents_demo.py model_client.py hw1_client.py
code > web_application > index.html > html
2 <html lang="en">
3 <head>
10 <style>
66
67     h1 {
68         font-family: 'Archivo', Arial, sans-serif;
69         font-weight: 800;
70         font-size: 2.15rem;
71         letter-spacing: -0.01em;
72         margin: 6px 0 28px;
73         color: var(--ink);
74     }
75
76     .route-strip {
77         display: flex;
78         height: 6px;
79         border-radius: 6px;
80         overflow: hidden;
81         margin-bottom: 0;
82     }
83     .route-strip span { flex: 1; }
84     .route-strip span:nth-child(1) { background: var(--delay); }
85     .route-strip span:nth-child(2) { background: var(--breakdown); }
86     .route-strip span:nth-child(3) { background: var(--accident); }
87     .route-strip span:nth-child(4) { background: var(--service-change); }
88
89     form {
90         background: var(--card);
91         border: 1px solid var(--line);
92         border-radius: 4px;
93         padding: 36px 34px 32px;
94         box-shadow: 0 18px 40px -20px rgba(22,33,62,0.25);
95     }
96
97     label {
98         display: flex;
99         align-items: baseline;
100        gap: 8px;
101        margin-top: 22px;
102        margin-bottom: 8px;
103        font-weight: 600;
104        font-size: 0.92rem;
105        color: var(--ink);
106    }
107    label:first-of-type { margin-top: 0; }
108
109    label .req {
110        font-family: 'IBM Plex Mono', monospace;
111        font-weight: 500;
112        font-size: 0.68rem;
113        letter-spacing: 0.06em;
114        text-transform: uppercase;
115        color: var(--amber);
116        background: rgba(242,169,60,0.14);
117        padding: 2px 6px;
118        border-radius: 3px;
119    }
120
121    input[type="text"],
122    input[type="email"],
123    textarea,
124    select {
125        width: 100%;
126        padding: 11px 13px;

```

```
index.html x agents_demo.py model_client.py hw1_client.py METRICS.md AI_USE.md M RUN_LOG.txt verification.json nondeterminism_input.json
code > web_application > index.html > html
2 <html lang="en">
3 <head>
10 <style>
121 input[type="text"],
122 input[type="email"],
123 textarea,
124 select {
125   width: 100%;
126   padding: 11px 13px;
127   border: 1.5px solid var(--line);
128   border-radius: 4px;
129   font-size: 0.98rem;
130   font-family: 'IBM Plex Sans', Arial, sans-serif;
131   color: var(--ink);
132   background: #F0FCFA;
133   transition: border-color 0.15s ease, box-shadow 0.15s ease;
134 }
135
136 #routeId {
137   font-family: 'IBM Plex Mono', monospace;
138   font-weight: 600;
139   letter-spacing: 0.02em;
140 }
141
142 input:focus, textarea:focus, select:focus {
143   outline: none;
144   border-color: var(--amber);
145   box-shadow: 0 0 0 3px #000000;
146 }
147
148 textarea {
149   resize: vertical;
150   min-height: 96px;
151   line-height: 1.5;
152 }
153
154 select {
155   appearance: none;
156   background-image: url("data:image/svg+xml;charset=UTF-8,%3Csvg xmlns='http://www.w3.org/2000/svg' width='14' height='9' viewBox='0 0 14 9'%3E%3Cpath d='M1 1L7 7L13 1' stroke='%23555' stroke-width='1'/%3E%3C/svg>");
157   background-repeat: no-repeat;
158   background-position: right 14px center;
159   padding-right: 36px;
160   cursor: pointer;
161 }
162
163 .checkbox-row {
164   display: flex;
165   align-items: center;
166   gap: 10px;
167   margin-top: 24px;
168   padding: 12px 14px;
169   background: #000000;
170   border-radius: 4px;
171 }
172
173 .checkbox-row input {
174   width: 17px;
175   height: 17px;
176   accent-color: var(--ink);
177   cursor: pointer;
178 }
179
180 .checkbox-row label {
181   margin: 0;
182   font-weight: 500;
```

```

index.html x  agents_demo.py  model_client.py  hw1_client.py  METRICS.md
code > web_application > index.html > html
2  <html lang="en">
3  <head>
10 <style>
178 .checkbox-row label {
182     color: var(--steel);
183     cursor: pointer;
184 }
185
186 button {
187     margin-top: 26px;
188     width: 100%;
189     padding: 14px 20px;
190     background: var(--ink);
191     color: #FFF;
192     border: none;
193     border-radius: 4px;
194     font-family: 'Archivo', Arial, sans-serif;
195     font-weight: 700;
196     font-size: 1rem;
197     letter-spacing: 0.01em;
198     cursor: pointer;
199     transition: background 0.15s ease, transform 0.1s ease;
200 }
201 button:hover { background: #0F1730; }
202 button:active { transform: scale(0.99); }
203
204 .footnote {
205     margin-top: 18px;
206     font-family: 'IBM Plex Mono', monospace;
207     font-size: 0.72rem;
208     color: var(--steel);
209     text-align: center;
210     letter-spacing: 0.03em;
211 }
212 </style>
213 </head>
214 <body>
215
216 <div class="page">
217   <div class="masthead">
218     <span class="line-dot"></span>
219     <span class="eyebrow">Route 2 &mdot; Rider Report Line</span>
220   </div>
221   <h1>Municipal Transit Incident Report</h1>
222
223   <div class="route-strip" aria-hidden="true">
224     <span></span><span></span><span></span><span></span><span></span>
225   </div>
226
227   <form id="incidentForm">
228     <label for="routeId">Route / Incident Title <span class="req">Required</span></label>
229     <input
230       type="text"
231       id="routeId"
232       name="routeId"
233       placeholder="e.g., Line 22 - Bus stalled on Main St"
234       required
235       autofocus
236     />
237
238     <label for="location">Location <span class="req">Required</span></label>
239     <input
240       type="text"
241       id="location"

```

```
index.html X agents_demo.py model_client.py hw1_client.py METRICS.md AI_USE.m...
code > web_application > index.html > html
2   <html lang="en">
214 <body>
216   <div class="page">
225     </div>
226
227   <form id="incidentForm">
228     <label for="routeId">Route / Incident Title <span class="req">Required</span></label>
229     <input
230       type="text"
231       id="routeId"
232       name="routeId"
233       placeholder="e.g., Line 22 - Bus stalled on Main St"
234       required
235       autofocus
236     />
237
238     <label for="location">Location <span class="req">Required</span></label>
239     <input
240       type="text"
241       id="location"
242       name="location"
243       placeholder="e.g., Main St & 5th Ave stop"
244       required
245     />
246
247     <label for="reporterEmail">Your Email <span class="req">Required</span></label>
248     <input
249       type="email"
250       id="reporterEmail"
251       name="reporterEmail"
252       placeholder="e.g., jane.doe@email.com"
253       required
254     />
255
256     <label for="description">Incident Description <span class="req">Required</span></label>
257     <textarea
258       id="description"
259       name="description"
260       placeholder="Describe what happened, when, and any other relevant details (min. 25 characters)"
261       required
262     ></textarea>
263
264     <label for="category">Category</label>
265     <select id="category" name="category">
266       <option value="Delay">Delay</option>
267       <option value="Breakdown">Breakdown</option>
268       <option value="Accident">Accident</option>
269       <option value="Service Change">Service Change</option>
270     </select>
271
272     <div class="checkbox-row">
273       <input type="checkbox" id="agreeToTerms" name="agreeToTerms" />
274       <label for="agreeToTerms">I agree to the terms and conditions.</label>
275     </div>
276
277     <button type="submit">Report Incident</button>
278   </form>
279
280   <div class="footnote">DATA-260 &middot; DOMAIN_ID 2 &middot; Municipal Transit Incidents</div>
281 </div>
282
283 <script src="script.js"></script>
284 </body>
285 </html>
```

Javascript:
script.js

```

code > web_application > JS script.js > ...
1  // ===== Part II: JavaScript =====
2
3  // 5. Closure to track how many times the form has been successfully submitted
4  const createSubmissionCounter = () => {
5      let count = 0;
6      return () => {
7          count += 1;
8          return count;
9      };
10 };
11 const trackSubmission = createSubmissionCounter();
12
13 // 1. Arrow function validation
14 const validateForm = (data) => {
15     // a) description must be more than 25 characters
16     if (!data.description || data.description.length <= 25) {
17         alert("Please enter a description with more than 25 characters.");
18         return false;
19     }
20
21     // b) terms and conditions checkbox must be checked
22     if (!data.agreeToTerms) {
23         alert("You must agree to the terms and conditions before submitting.");
24         return false;
25     }
26
27     return true;
28 };
29
30 document.getElementById("incidentForm").addEventListener("submit", (event) => {
31     event.preventDefault();
32
33     const form = event.target;
34     const formData = {
35         routeId: form.routeId.value,
36         location: form.location.value,
37         reporterEmail: form.reporterEmail.value,
38         description: form.description.value,
39         category: form.category.value,
40         agreeToTerms: form.agreeToTerms.checked,
41     };
42
43     if (!validateForm(formData)) {
44         return;
45     }
46
47     // 2. Convert form data into a JSON string and log it
48     const jsonString = JSON.stringify(formData);
49     console.log("Form data as JSON string:", jsonString);
50
51     // Parse it back into an object to work with below
52     const parsedData = JSON.parse(jsonString);
53
54     // 3. Object destructuring to extract primary field and email, then log them
55     const { routeId, reporterEmail } = parsedData;
56     console.log("Primary field (routeId):", routeId);
57     console.log("Email field (reporterEmail):", reporterEmail);
58
59     // 4. Spread operator to add submissionDate to the parsed object
60     const updatedData = {
61         ...parsedData,
62         submissionDate: new Date().toISOString(),
63     };
64     console.log("Updated parsed object with submissionDate:", updatedData);
65
66     // 5. Log the submission count using the closure
67     const submissionCount = trackSubmission();
68     console.log("Total successful submissions this session:", submissionCount);
69
70     alert("Incident reported successfully!");
71     form.reset();
72     form.routeId.focus();
73 });

```

ROUTE 2 - RIDER REPORT LINE

Municipal Transit Incident Report

Route / Incident Title REQUIRED

e.g., Line 22 - Bus stalled on Main St

Location REQUIRED

e.g., Main St & 5th Ave stop

Your Email REQUIRED

e.g., jane.doe@email.com

Incident Description REQUIRED

Describe what happened, when, and any other relevant details (min. 25 characters)

Category

Delay

☐ I agree to the terms and conditions.

Report Incident

DATA-260 - DOMAIN_ID 2 - Municipal Transit Incidents

Fig 1 Web Page

ROUTE 2 - RIDER REPORT LINE

Municipal Transit Incident Report

Route / Incident Title REQUIRED

sunnyvale

Location REQUIRED

3rd street

Your Email REQUIRED

ab

Incident Description REQUIRED

Please enter a description with more than 25 characters.

Close

Category

Delay

☒ I agree to the terms and conditions.

Report Incident

DATA-260 - DOMAIN_ID 2 - Municipal Transit Incidents

Fig 2 An alert messaged will popped if you characters are less than 25

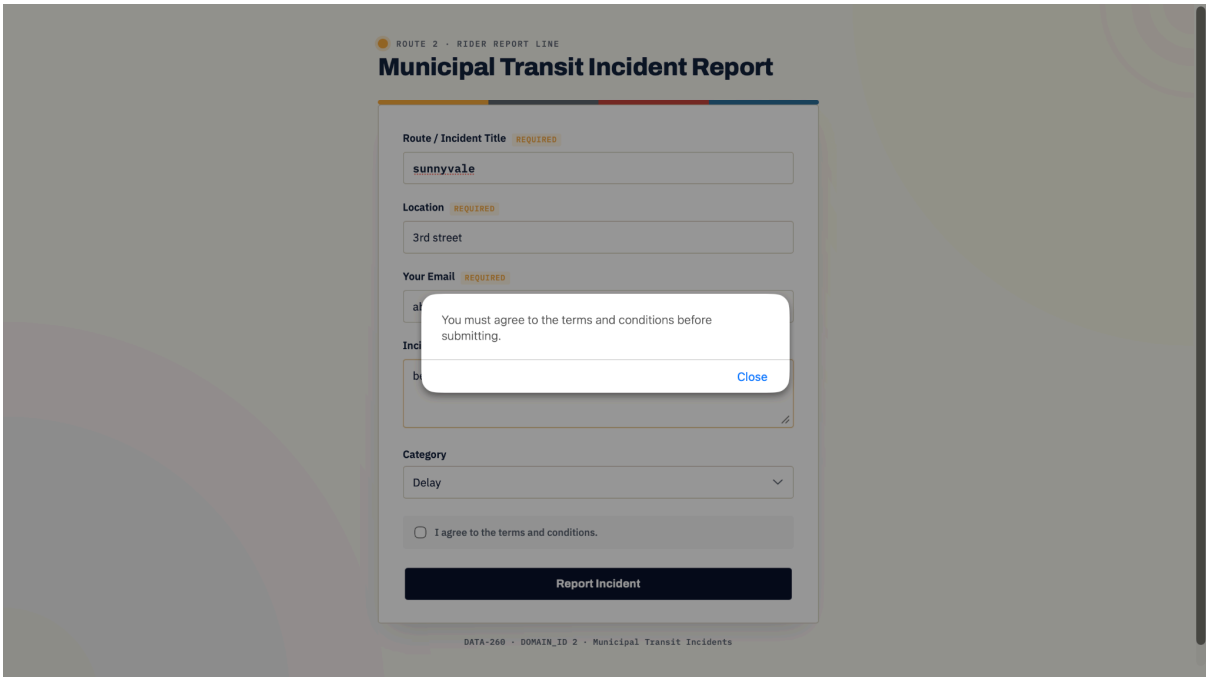


Fig 3 If you dont checked the terms and condition check box the alert message will popped

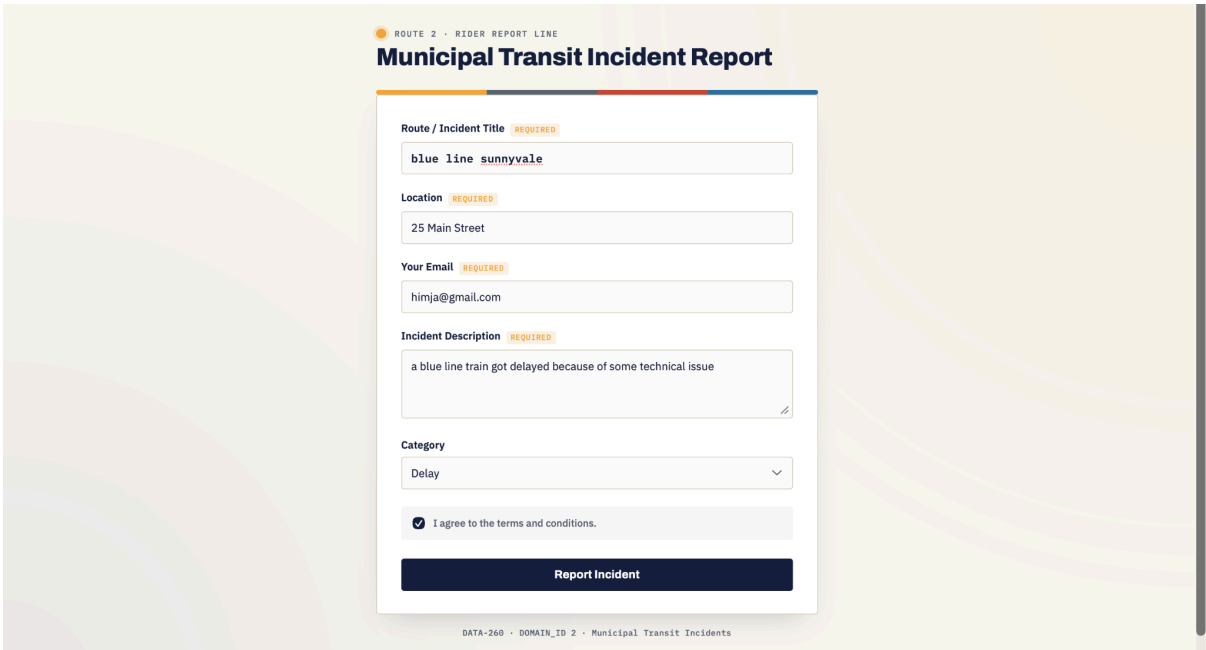


Fig 4 Entering values

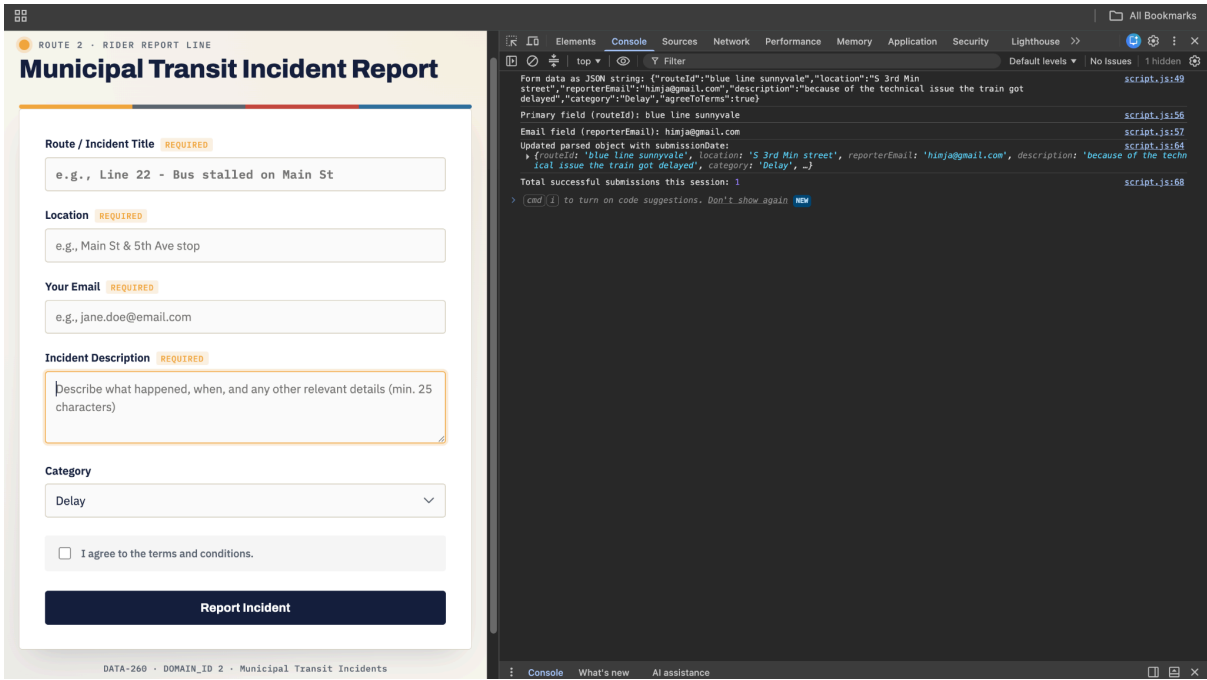


Fig 5 console before submission

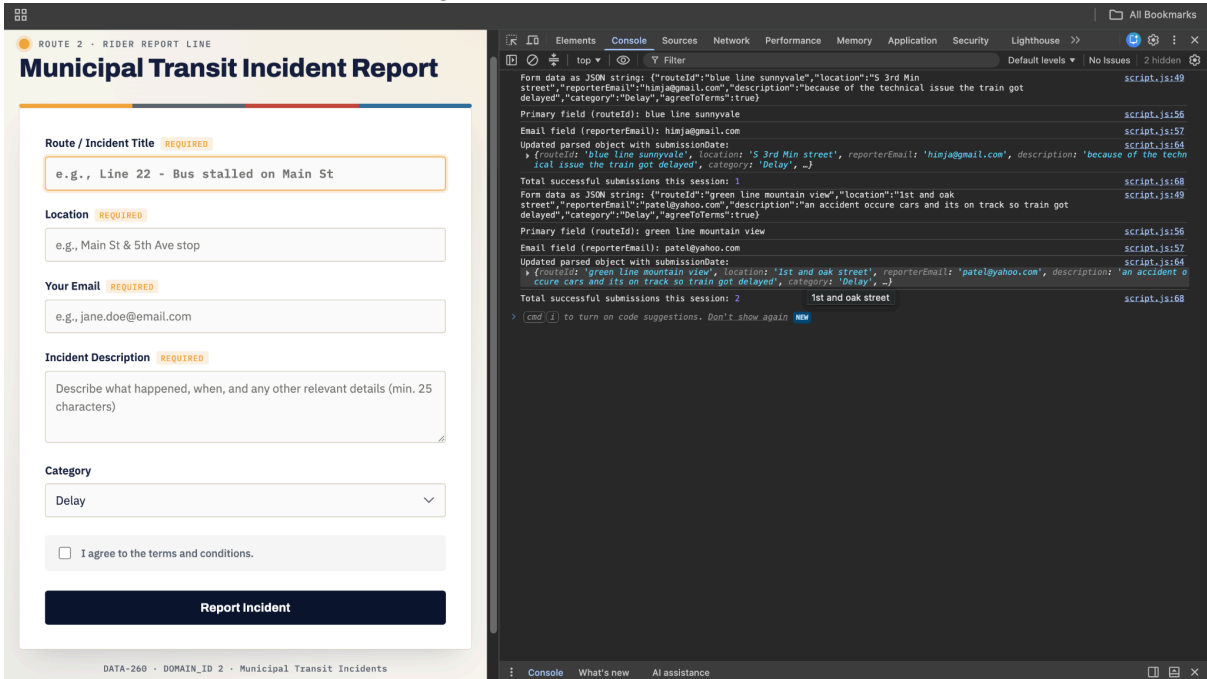


Fig 6 console after 2 successful submission

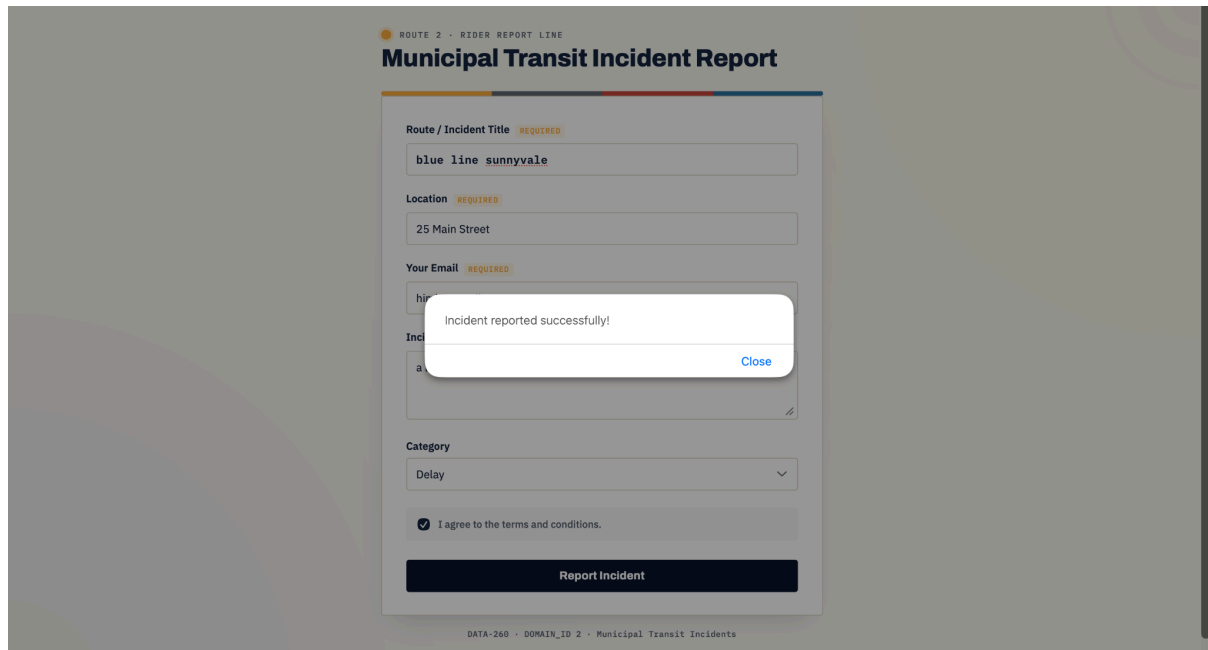


Fig 7 Report successfully submitted

Part 2 JavaScript features implemented (code/web_application/script.js)

- Arrow-function form validation (description > 25 characters, terms checkbox required) with alert-based feedback
- JSON.stringify of the submitted form data logged to console on successful submit
- Object destructuring to pull out the primary field and email for logging
- Spread operator used to add a submissionDate field to a copy of the form data
- A closure (createSubmissionCounter) tracking how many times the form has been successfully submitted

Part 3

Docker and AWS Deployment

Docker (local, port 8446)

The application was containerized with a Dockerfile serving the static site via nginx on port 8446

(PORT_BASE), and ran locally via `docker run -p 8446:8446` before being pushed to Amazon ECR.

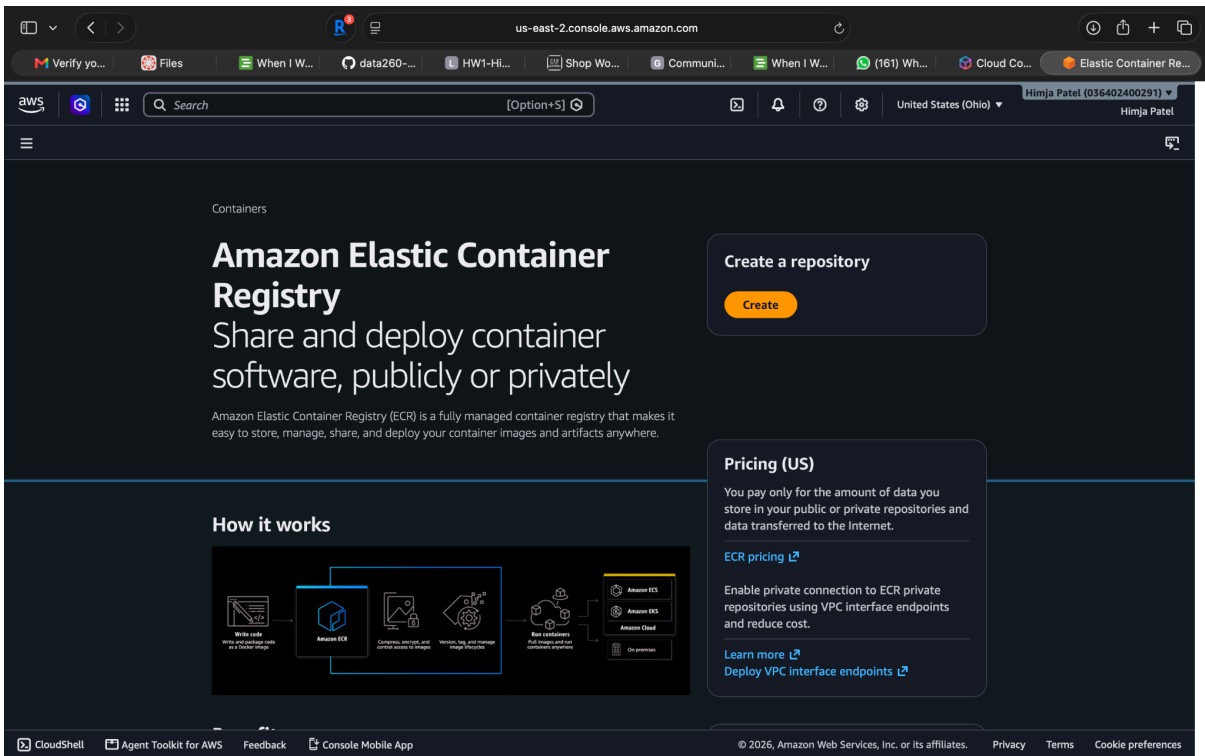


Fig 8 Amazon ECR landing page, beginning repository creation.

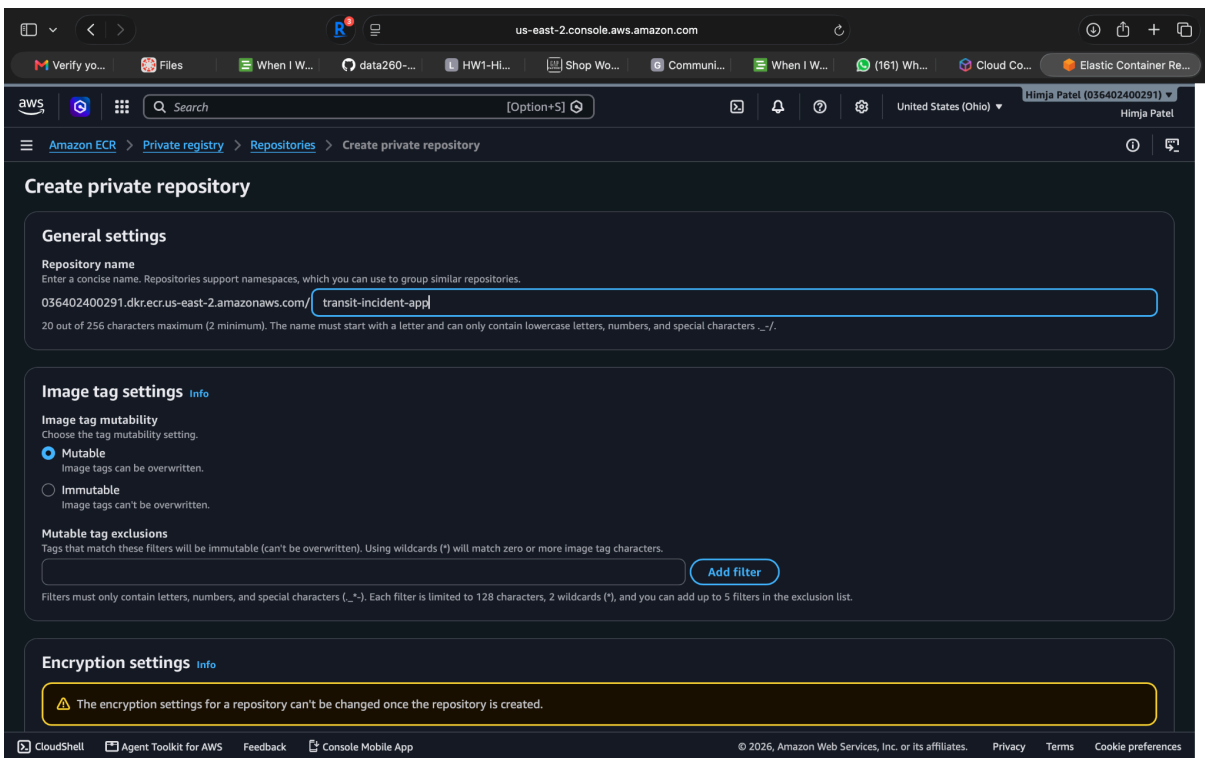


Fig 9 Creating Private Repository

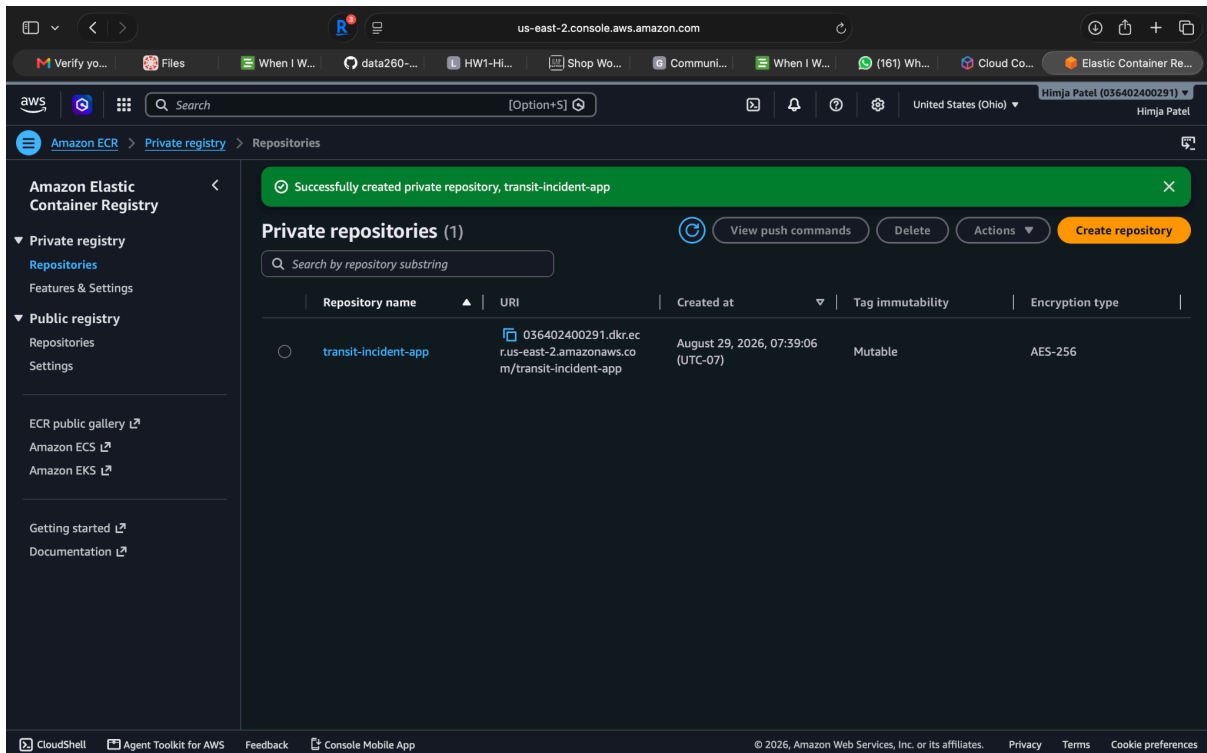


Fig 10 Repository created successfully

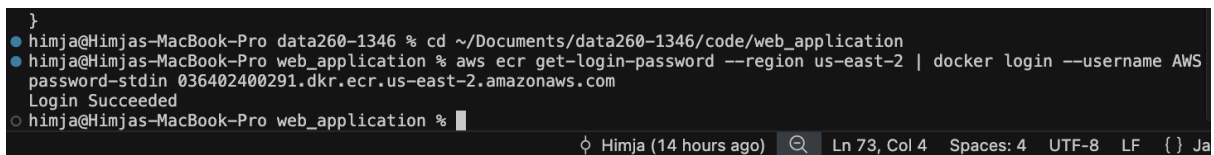


Fig 11 AWS CLI authenticated and docker login to ECR succeed

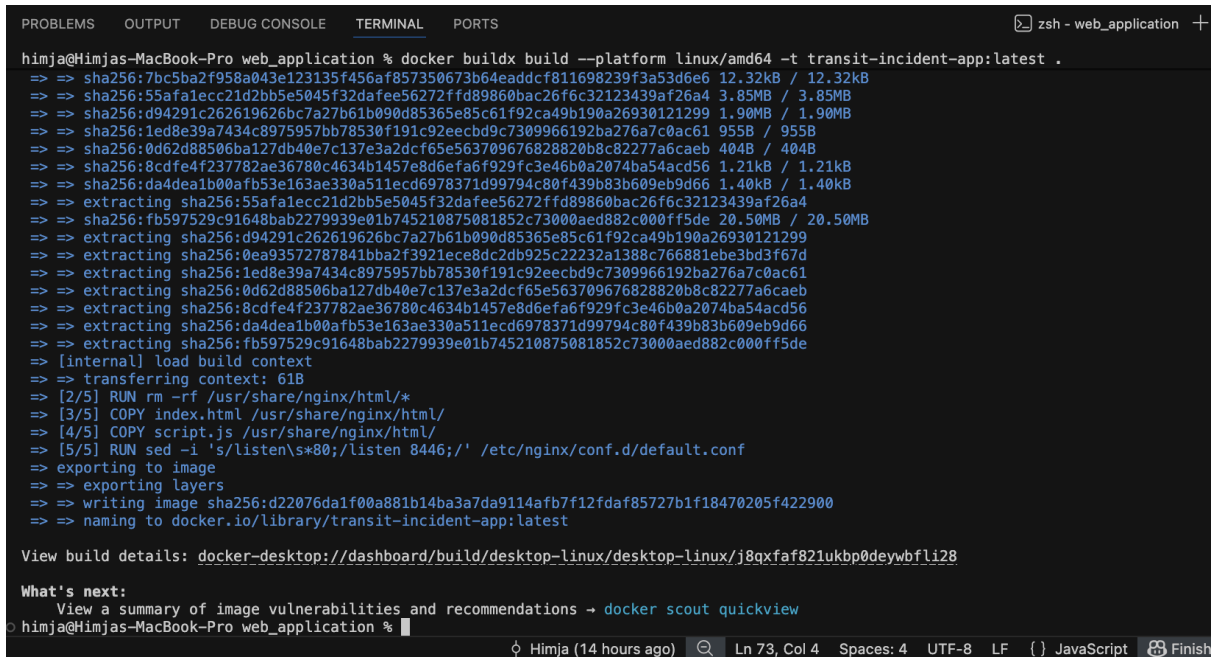


Fig 12 Building Docker Image

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
himja@Himjas-MacBook-Pro web_application % docker buildx build --platform linux/amd64 -t transit-incident-app:latest .
=> transferring context: 61B
=> [2/5] RUN rm -rf /usr/share/nginx/html/*
=> [3/5] COPY index.html /usr/share/nginx/html/
=> [4/5] COPY script.js /usr/share/nginx/html/
=> [5/5] RUN sed -i 's/listen\s*80;/listen 8446;/' /etc/nginx/conf.d/default.conf
=> exporting to image
=> exporting layers
=> writing image sha256:d22076da1f00a881b14ba3a7da9114afb7f12daf85727b1f18470205f422900
=> naming to docker.io/library/transit-incident-app:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/j8qxfaf821ukbp0deywbfli28

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview
himja@Himjas-MacBook-Pro web_application % docker tag transit-incident-app:latest 036402400291.dkr.ecr.us-east-2.amazonaws.com/transit-incident-app:latest
himja@Himjas-MacBook-Pro web_application % docker push 036402400291.dkr.ecr.us-east-2.amazonaws.com/transit-incident-app:latest
The push refers to repository [036402400291.dkr.ecr.us-east-2.amazonaws.com/transit-incident-app]
66af86664155: Pushed
d69984d4d4dae: Pushed
22713164c4cb: Pushed
11988a7669f7: Pushed
edc23e36f952: Pushed
628c86fdf4c1: Pushed
63c800442e2a: Pushed
31e777702b00: Pushed
e3b86476d58e: Pushed
18ca587a5d44: Pushed
1c4d71cf6bfe: Pushed
34884abb928: Pushed
latest: digest: sha256:4c0812aa650e20ca2658b51aa4b0275d55ca95945c24059e5ac099201eaf314 size: 2819
himja@Himjas-MacBook-Pro web_application %

```

Fig 13 Pushing image to ECR

ECS cluster, task definition, and service

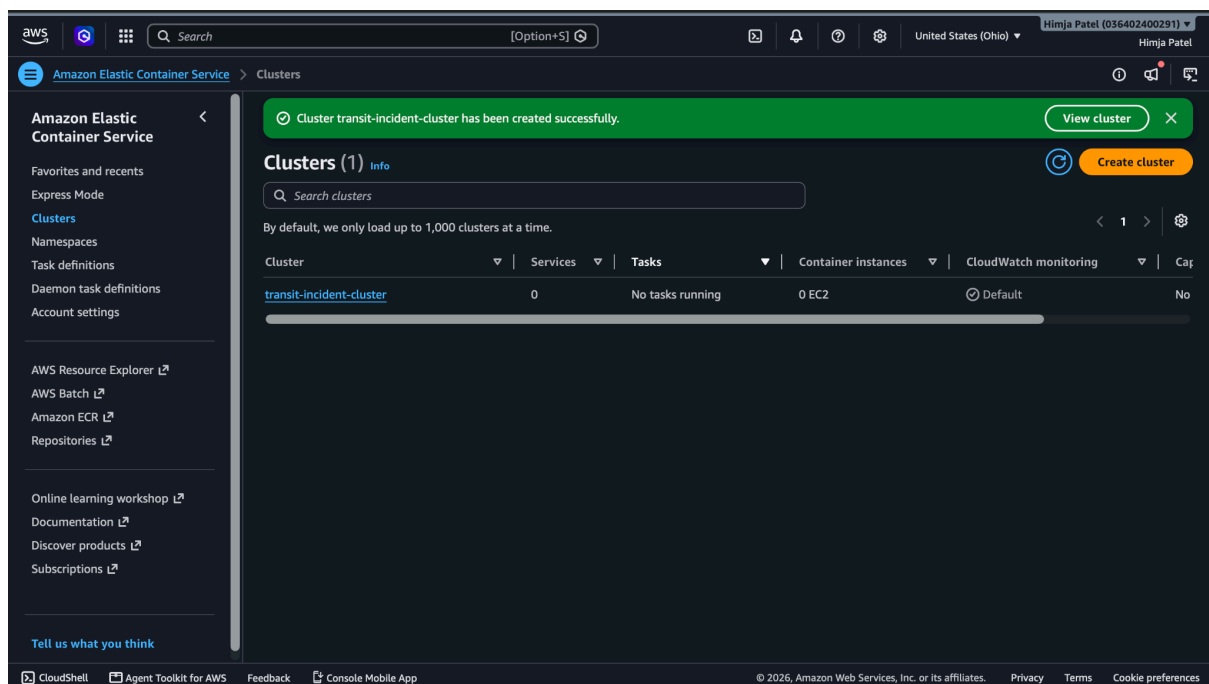


Fig 14 ECS cluster transit-incident-cluster created successfully

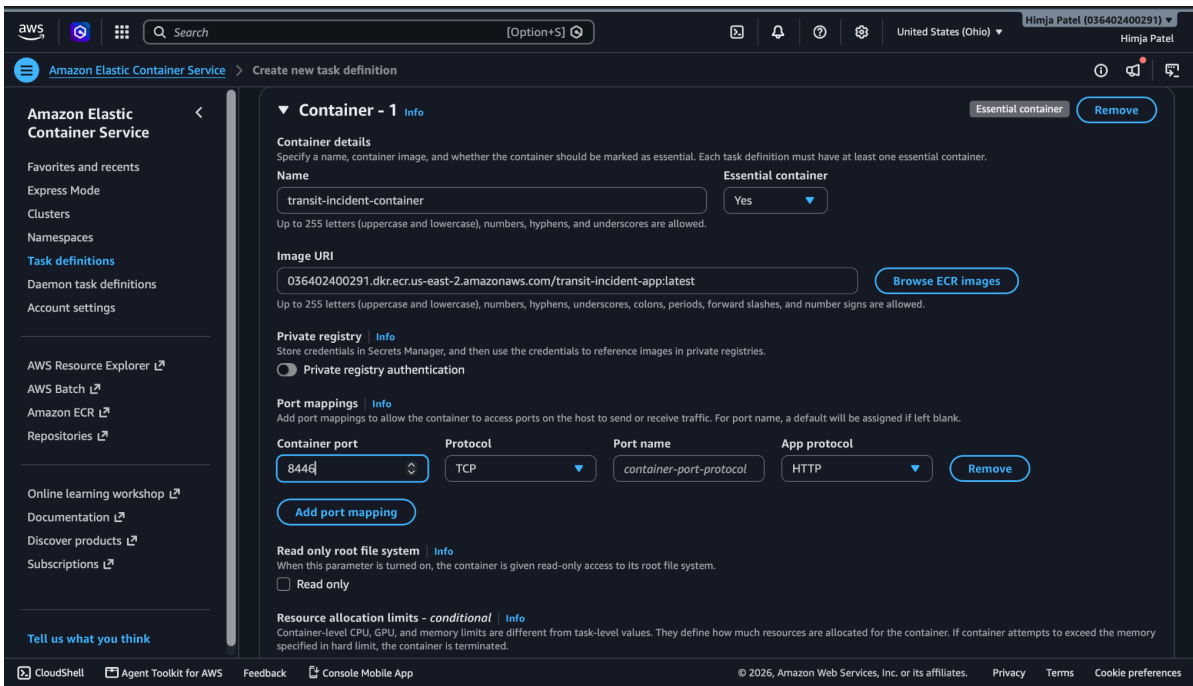


Fig 15 Task definition container port 8446 mapped, TCP/HTTP

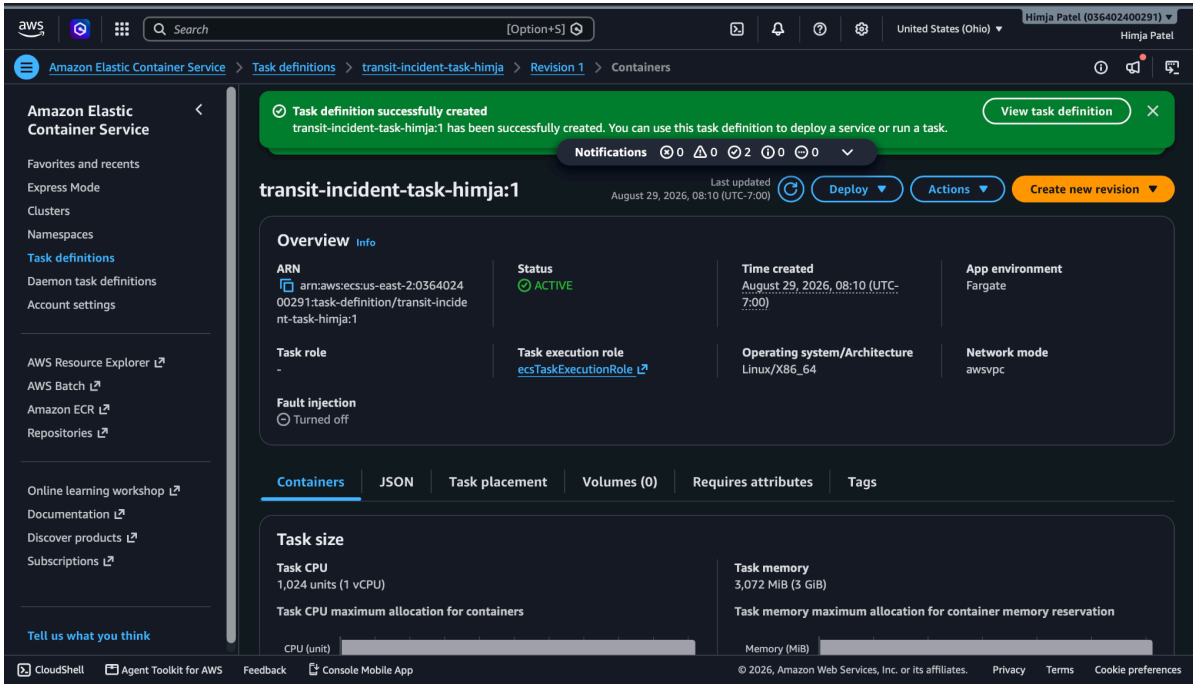


Fig 16 Task definition successfully created

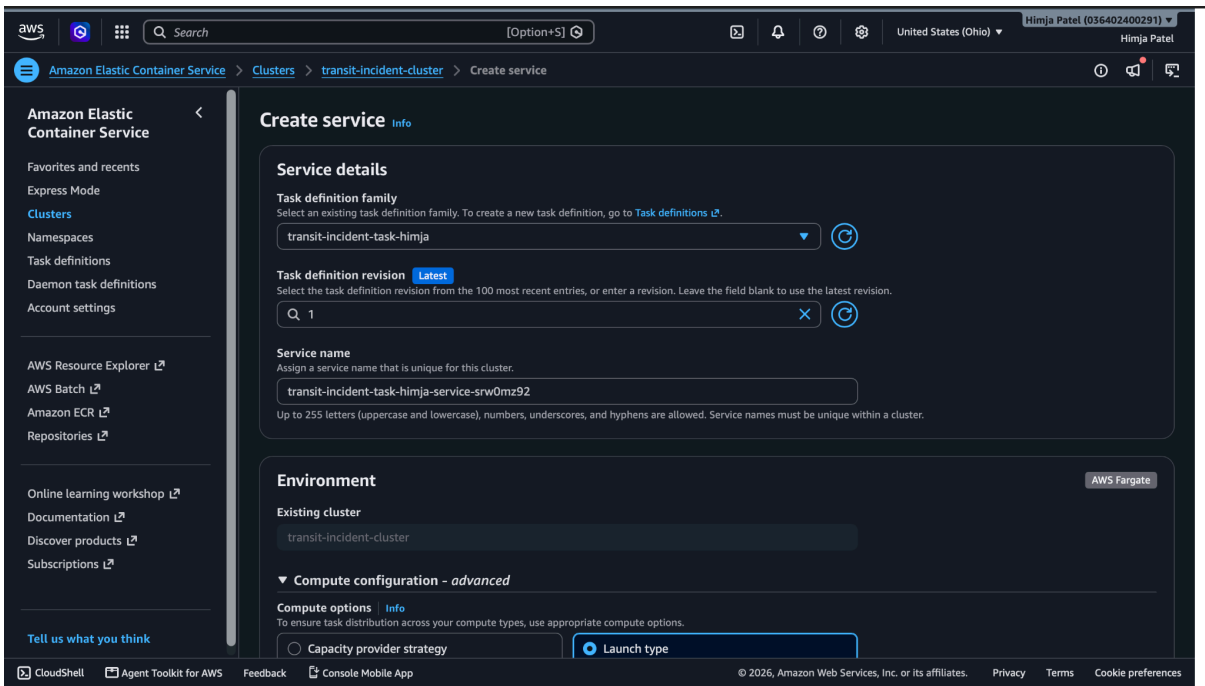


Fig 17 Creating ECS service from task definition

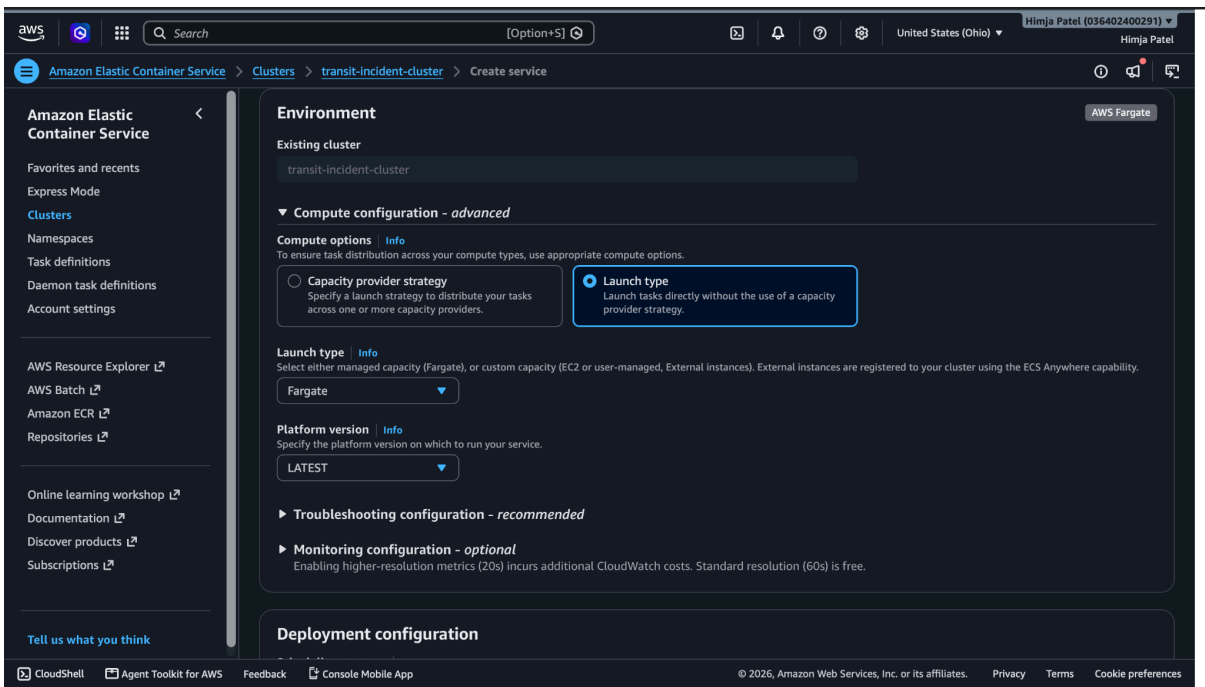


Fig 18 Service environment

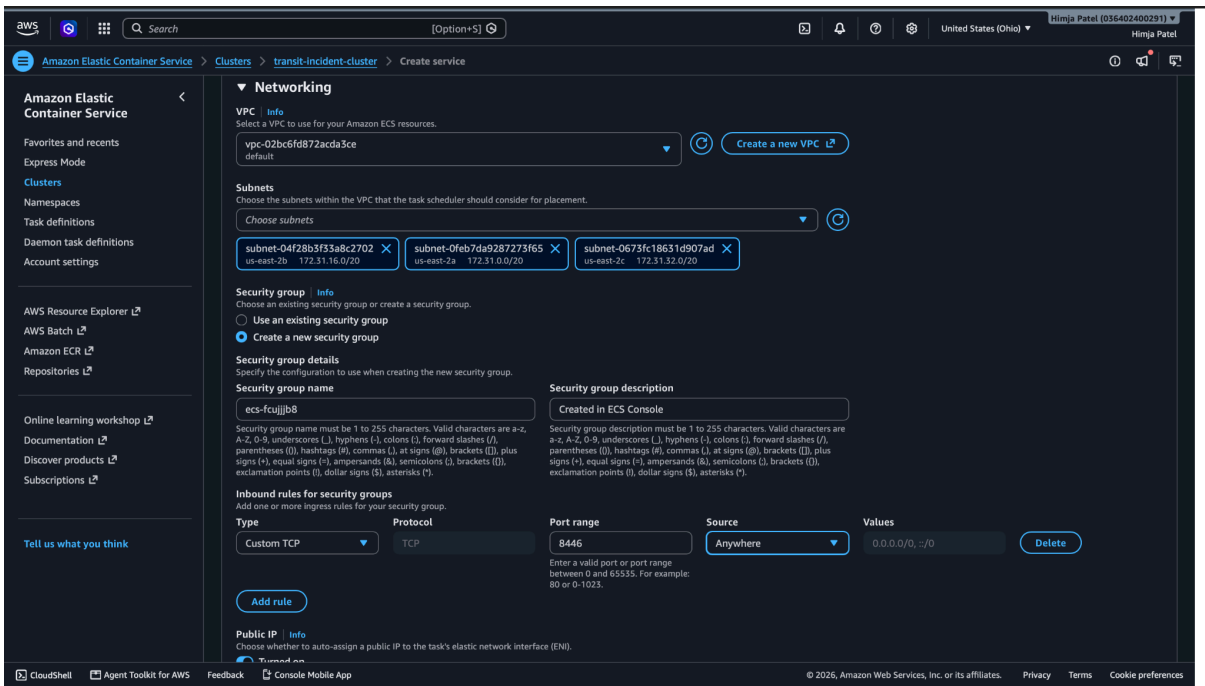


Fig 19 Networking -VPC, subnets, and a new security group allowing inbound TCP 8446 from anywhere

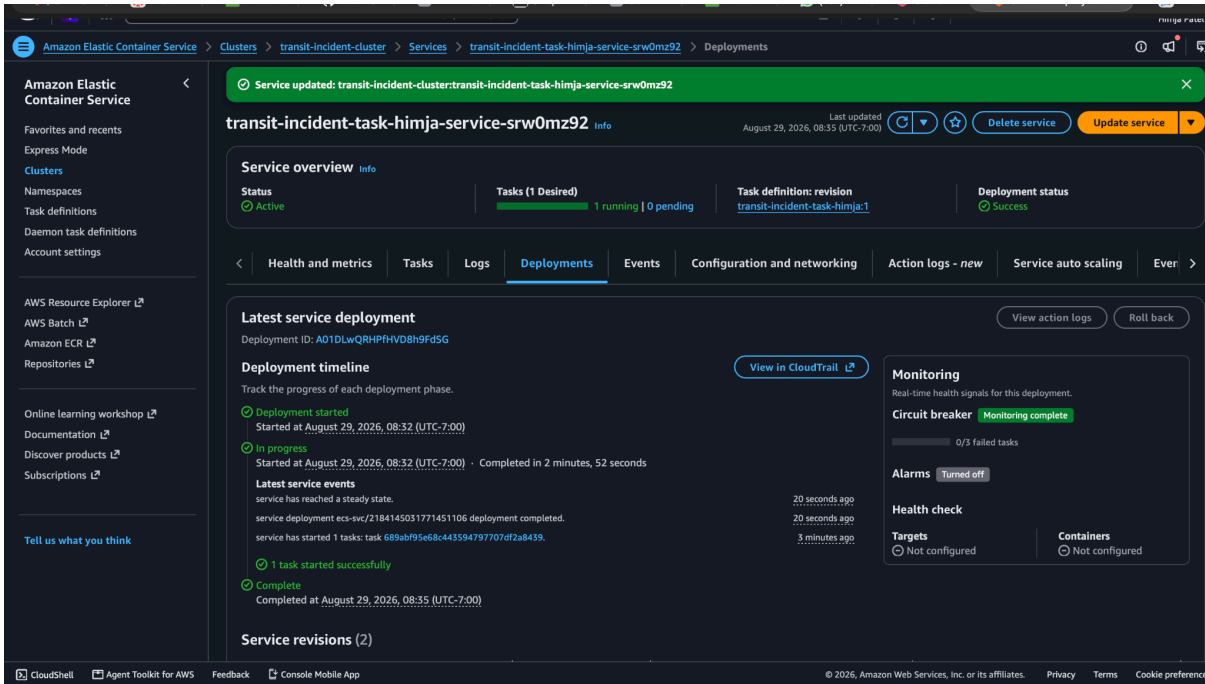


Fig 20 Service deployment completed successfully

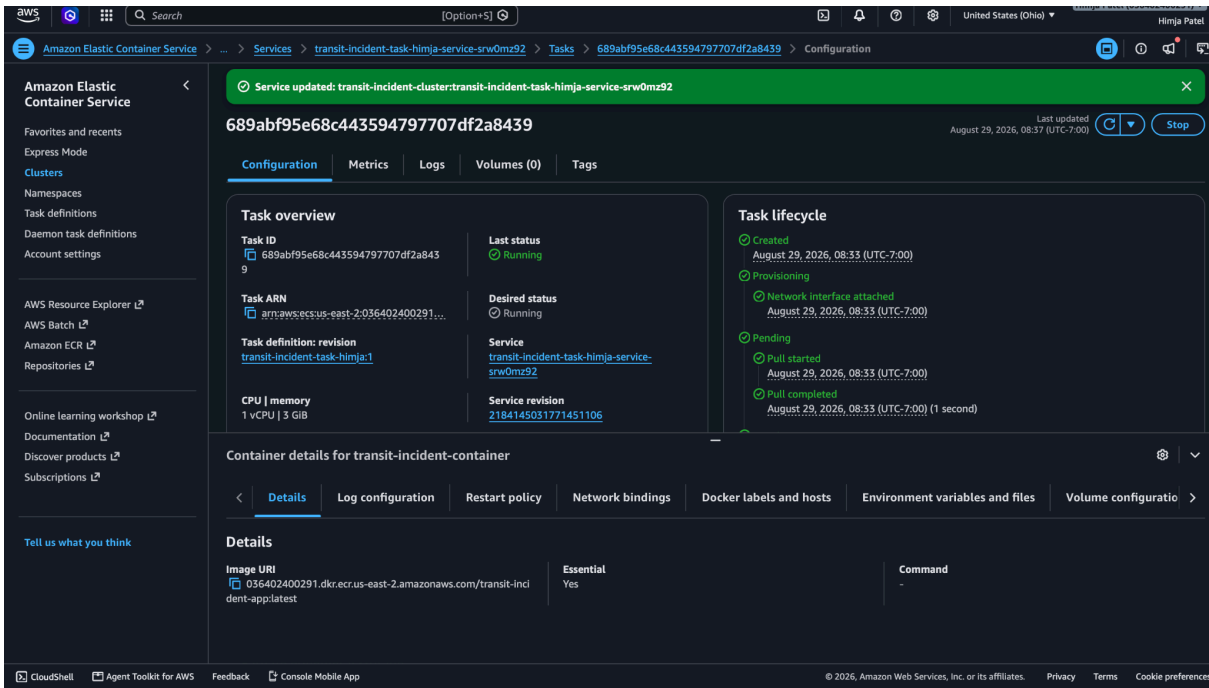


Fig 21 Running task configuration conforming the image URI

Public IP Verification

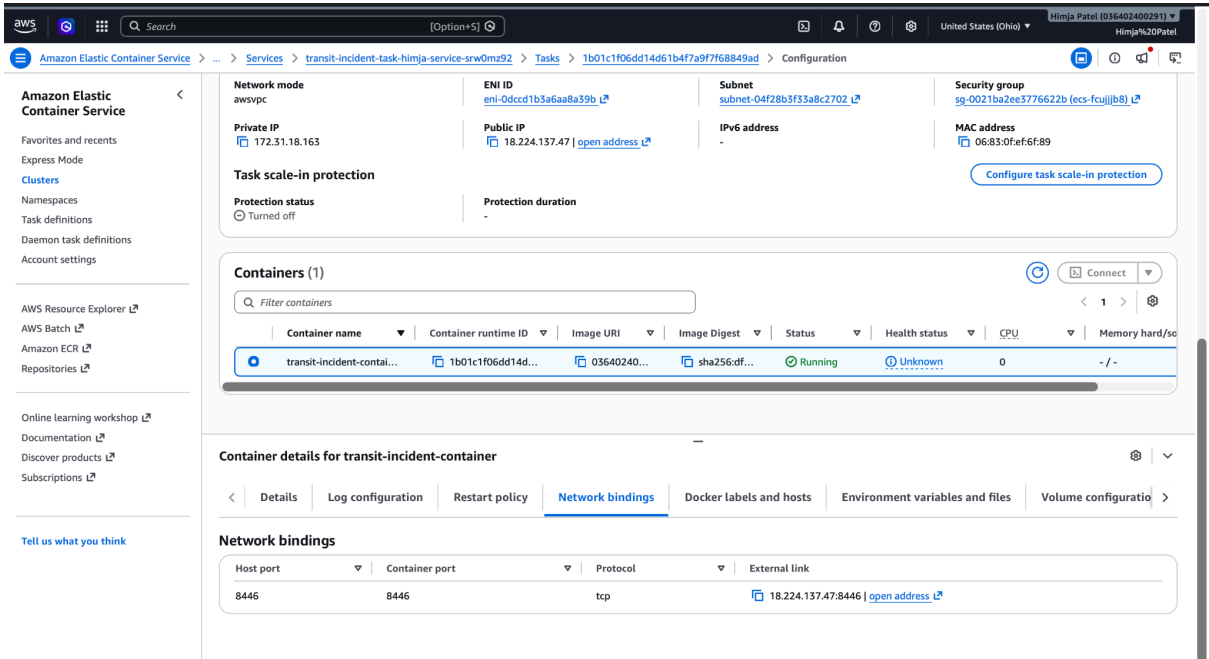
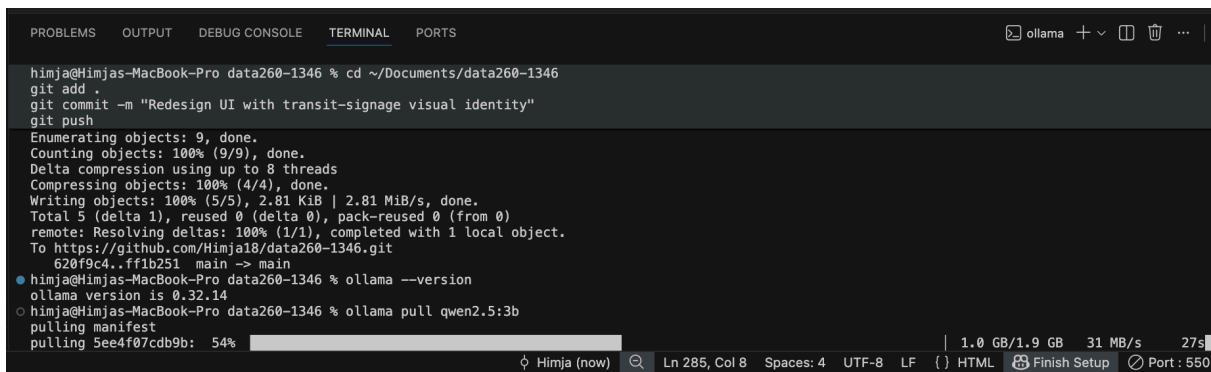


Fig 22 Network Building public endpoint 18.224.137.47:8446(open Address)

Part 4: Ollama Agents

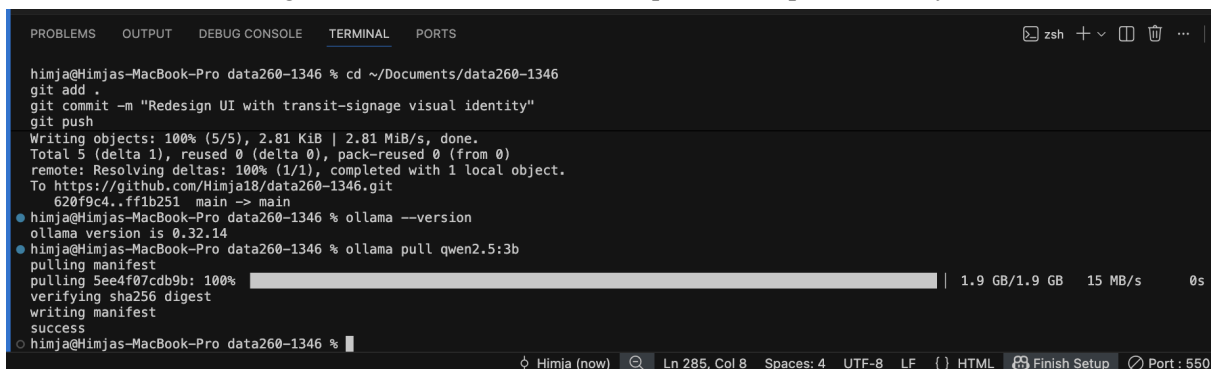


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
himja@Himjas-MacBook-Pro data260-1346 % cd ~/Documents/data260-1346
git add
git commit -m "Redesign UI with transit-signage visual identity"
git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 2.81 KiB | 2.81 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Himja18/data260-1346.git
620f9c4..ff1b251 main -> main
himja@Himjas-MacBook-Pro data260-1346 % ollama --version
ollama version is 0.32.14
himja@Himjas-MacBook-Pro data260-1346 % ollama pull qwen2.5:3b
pulling manifest
pulling See4f07cdb9b: 54%
1.0 GB/1.9 GB 31 MB/s 27s

```

Fig 23 Ollama was installed and qwen2.5:3b pulled locally

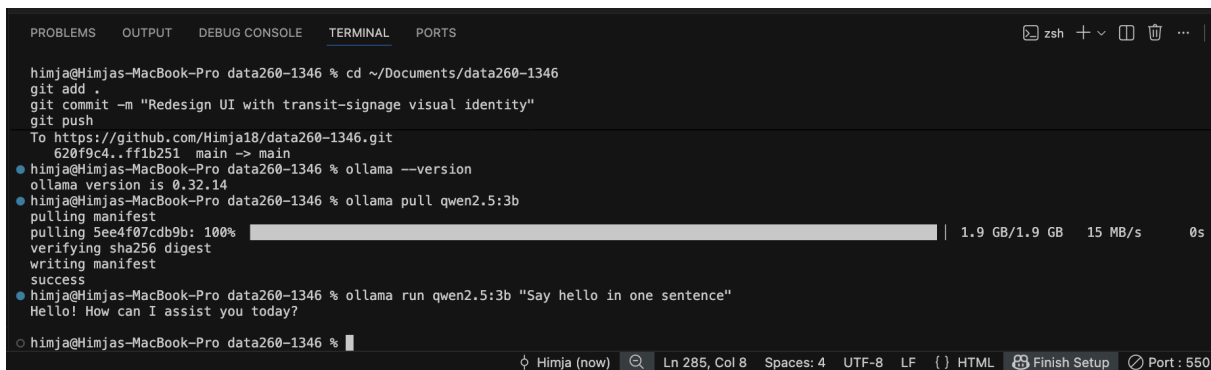


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
himja@Himjas-MacBook-Pro data260-1346 % cd ~/Documents/data260-1346
git add .
git commit -m "Redesign UI with transit-signage visual identity"
git push
Writing objects: 100% (5/5), 2.81 KiB | 2.81 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Himja18/data260-1346.git
620f9c4..ff1b251 main -> main
himja@Himjas-MacBook-Pro data260-1346 % ollama --version
ollama version is 0.32.14
himja@Himjas-MacBook-Pro data260-1346 % ollama pull qwen2.5:3b
pulling manifest
pulling See4f07cdb9b: 100%
verifying sha256 digest
writing manifest
success
himja@Himjas-MacBook-Pro data260-1346 %
1.9 GB/1.9 GB 15 MB/s 0s

```

Fig 24 Model pull completed successfully (1.9 GB)



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
himja@Himjas-MacBook-Pro data260-1346 % cd ~/Documents/data260-1346
git add .
git commit -m "Redesign UI with transit-signage visual identity"
git push
To https://github.com/Himja18/data260-1346.git
620f9c4..ff1b251 main -> main
himja@Himjas-MacBook-Pro data260-1346 % ollama --version
ollama version is 0.32.14
himja@Himjas-MacBook-Pro data260-1346 % ollama pull qwen2.5:3b
pulling manifest
pulling See4f07cdb9b: 100%
verifying sha256 digest
writing manifest
success
himja@Himjas-MacBook-Pro data260-1346 % ollama run qwen2.5:3b "Say hello in one sentence"
Hello! How can I assist you today?
himja@Himjas-MacBook-Pro data260-1346 %
1.9 GB/1.9 GB 15 MB/s 0s

```

Fig 25 Smoke-testing the model with ollama run qwen2.5:3b

agents_demo.py

agents_demo.py implements a three-stage pipeline over a single incident report: a Planner proposes tags and a summary, a Reviewer checks and can revise them, and a Finalizer guarantees the published output always has exactly 3 tags (backfilling from the Planner's list if the Reviewer trims below 3 — see AI_USE.md for the bug this fixed).

Code:

agents_demo.py

```

code > agents_demo.py > ...
21 import argparse
22 import json
23 import re
24 import sys
25 import time
26
27 from langchain_ollama import ChatOllama
28
29 MODEL_NAME = "qwen2.5:3b"
30
31
32 def build_llm(temperature: float) -> ChatOllama:
33     """Create a ChatOllama client at the given temperature."""
34     return ChatOllama(model=MODEL_NAME, temperature=temperature)
35
36
37 def extract_json(text: str) -> dict:
38     """
39     Best-effort extraction of a JSON object from an LLM response.
40     Local models sometimes wrap JSON in prose or code fences, so we
41     look for the first {...} block and parse that.
42     """
43     text = text.strip()
44     # Strip markdown code fences if present
45     text = re.sub(r"```(?:json)?\s*", "", text)
46     text = re.sub(r"\s*```\s*", "", text)
47
48     match = re.search(r"({.*})", text, re.DOTALL)
49     if not match:
50         raise ValueError(f"No JSON object found in model output:\n{text}")
51     return json.loads(match.group(0))
52
53
54 def run_planner(llm: ChatOllama, title: str, content: str) -> dict:
55     """
56     Planner agent: reads the raw title/content and proposes an initial
57     set of 3 tags and a draft summary (<=25 words). Tags/summary must be
58     derived from the actual text, not hardcoded domain keywords.
59     """
60     prompt = f"""You are the PLANNER agent in a content-tagging pipeline.
61
62 Given the TITLE and CONTENT below, produce:
63 1. Exactly 3 short topical tags (2-3 words each) that best describe the
64 specific content – derive them from what is actually written, do not
65 invent generic categories that aren't supported by the text.
66 2. A one-sentence summary of at most 25 words.
67
68 TITLE: {title}
69 CONTENT: {content}
70
71 Respond with ONLY a JSON object in this exact shape, no other text:
72 {{{"tags": ["tag1", "tag2", "tag3"], "summary": "..."}}}
73 """
74     response = llm.invoke(prompt)
75     return extract_json(response.content)
76
77
78 def run_reviewer(llm: ChatOllama, title: str, content: str, planner_output: dict) -> dict:
79     """
80     Reviewer agent: critiques the Planner's draft against the source text
81     and either approves it as-is or returns a corrected version. This is
82     what lets us answer "did the Reviewer change anything?" honestly.
83     """
84     prompt = f"""You are the REVIEWER agent in a content-tagging pipeline.
85
86 Original TITLE: {title}
87 Original CONTENT: {content}
88
89 The PLANNER produced this draft:
90 {json.dumps(planner_output)}
91
92 Check the draft against the original text:
93 - Are all 3 tags actually supported by the content (not generic/invented)?
94 - Is the summary accurate and at most 25 words?
95 - Are there exactly 3 tags, no more, no fewer?
96
97 If the draft is good, return it unchanged. If not, return a corrected
98 version. Either way, respond with ONLY a JSON object in this exact shape,
99 no other text:
100 {{{"tags": ["tag1", "tag2", "tag3"], "summary": "...", "changed": true/false, "review_notes": "one short sentence explaining what you checked or changed"}}}
101 """
102     response = llm.invoke(prompt)
103     return extract_json(response.content)
104
105
106 def finalize(reviewer_output: dict, planner_output: dict) -> dict:
107     """
108     Finalization step (deterministic, no LLM call): assembles the final
109     Publish-ready JSON from the Reviewer's output, enforcing the 3-tag /
110     25-word constraints as a hard safety net.
111
112 If the Reviewer returned fewer than 3 tags (a real failure mode we
113 observed in testing), backfill missing slots from the Planner's
114 original tags rather than silently publishing an under-filled list.
115 """

```

```

code > agents_demo.py > ...
106 def finalize(reviewer_output: dict, planner_output: dict) -> dict:
107     """
108     tags = list(dict.fromkeys(reviewer_output.get("tags", []))) # de-dupe, preserve order
109
110     if len(tags) < 3:
111         for t in planner_output.get("tags", []):
112             if t not in tags:
113                 tags.append(t)
114             if len(tags) == 3:
115                 break
116
117     tags = tags[:3]
118
119     summary = reviewer_output.get("summary", "")
120     words = summary.split()
121     if len(words) > 25:
122         summary = " ".join(words[:25])
123
124     return {
125         "tags": tags,
126         "summary": summary,
127     }
128
129 def run_pipeline(title: str, content: str, temperature: float, verbose: bool = True) -> dict:
130     """Runs the full Planner -> Reviewer -> Finalizer flow once and
131     returns a dict with timing info and all intermediate outputs."""
132     llm = build_llm(temperature)
133
134     start = time.time()
135
136     planner_output = run_planner(llm, title, content)
137     if verbose:
138         print("\n=== PLANNER OUTPUT ===")
139         print(json.dumps(planner_output, indent=2))
140
141     reviewer_output = run_reviewer(llm, title, content, planner_output)
142     if verbose:
143         print("\n=== REVIEWER OUTPUT ===")
144         print(json.dumps(reviewer_output, indent=2))
145
146     final_output = finalize(reviewer_output, planner_output)
147     latency_ms = (time.time() - start) * 1000
148
149     if verbose:
150         print("\n=== FINALIZED / PUBLISH OUTPUT ===")
151         print(json.dumps(final_output, indent=2))
152         print(f"\nlatency: {latency_ms:.0f} ms, temperature: {temperature}")
153
154     return {
155         "planner": planner_output,
156         "reviewer": reviewer_output,
157         "final": final_output,
158         "latency_ms": latency_ms,
159         "temperature": temperature,
160     }
161
162 def main():
163     parser = argparse.ArgumentParser(description="Planner -> Reviewer -> Finalizer tagging pipeline")
164     parser.add_argument("--title", type=str, help="Incident title")
165     parser.add_argument("--content", type=str, help="Incident content/description")
166     parser.add_argument("--input-file", type=str, help="Path to a JSON file with {'title':..., 'content':...}")
167     parser.add_argument("--temperature", type=float, default=0.7, help="Sampling temperature (default 0.7)")
168     parser.add_argument("--quiet", action="store_true", help="Suppress intermediate step printing (used for batch runs)")
169     args = parser.parse_args()
170
171     if args.input_file:
172         with open(args.input_file, "r") as f:
173             data = json.load(f)
174             title, content = data["title"], data["content"]
175     elif args.title and args.content:
176         title, content = args.title, args.content
177     else:
178         # Default demo input for a quick manual run
179         title = "Line 22 - Bus stalled on Main St"
180         content = {
181             "The 8:15 AM bus on Route 22 broke down near the Main St and "
182             "5th Ave intersection. Passengers were stranded for 20 minutes "
183             "before a replacement vehicle arrived. Driver reported a "
184             "mechanical issue with the engine."
185         }
186
187     result = run_pipeline(title, content, args.temperature, verbose=not args.quiet)
188
189     if args.quiet:
190         # For batch/non-determinism runs: print one JSON line per run
191         print(json.dumps({
192             "tags": result["final"]["tags"],
193             "summary": result["final"]["summary"],
194             "latency_ms": result["latency_ms"],
195             "temperature": result["temperature"],
196         }))
197
198
199
200
201
202
203
204
205
206
207
208

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) himja@Himjas-MacBook-Pro code % python agents_demo.py
{"tags": ["Intersection"], "summary": "8:15 AM bus on Route 22 stalls due to engine failure near Main St and 5th Ave."}

(latency: 19605 ms, temperature: 0.7)
(venv) himja@Himjas-MacBook-Pro code % python agents_demo.py

=== PLANNER OUTPUT ===
{"tags": ["BusDelay", "MechanicalIssue", "Intersection"], "summary": "8:15 AM bus on Route 22 stalls near Main St and 5th Ave, passengers wait 20 minutes."}

=== REVIEWER OUTPUT ===
{"tags": ["BusDelay", "MechanicalIssue", "Intersection"], "summary": "8:15 AM bus on Route 22 stalls near Main St and 5th Ave, passengers wait 20 minutes.", "changed": true, "review_notes": "Removed 'Route 22' from tags as it's not directly supported by the content."}

=== FINALIZED / PUBLISH OUTPUT ===
{"tags": ["BusDelay", "MechanicalIssue", "Intersection"], "summary": "8:15 AM bus on Route 22 stalls near Main St and 5th Ave, passengers wait 20 minutes."}

(latency: 16037 ms, temperature: 0.7)
(venv) himja@Himjas-MacBook-Pro code %

```

Fig 26 agents_demo.py output

Required Questions:

1. Final 3 tags: BusDelay, MechanicalIssue, Intersection
2. Summary ≤ 25 words: 8:15 AM bus on Route 22 stalls due to engine failure near Main St and 5th Ave."
3. Did the reviewer change anything? : Yes, in general across multiple runs the Reviewer actively modified tags (e.g. dropping an unsupported tag) in some cases and passed tags through unchanged with review_notes explaining why in others, confirming it performs real review rather than rubber-stamping the Planner's output.

Part 5: Non determinism experiment

The pipeline was run 20 times at temperature 0.7 and 20 times at temperature 0.0 on a single fixed input ("Line 22 - Bus stalled on Main St"), for 40 runs total. Raw results are saved to reports/hw01/raw/nondeterminism_raw.json / .csv, with the computed summary in reports/hw01/raw/nondeterminism_summary.json and reports/hw01/METRICS.md.

```

○ (venv) himja@Himjas-MacBook-Pro data260-1346 % cd code
python run_nondeterminism_experiment.py
Fixed input: 'Line 22 - Bus stalled on Main St'
Running 20 runs at temperature=0.7 ...
[temp=0.7] run 1/20...
[temp=0.7] run 2/20...
[temp=0.7] run 3/20...
[temp=0.7] run 4/20...
[temp=0.7] run 5/20...
[temp=0.7] run 6/20...

```

Fig 27 Batch running in progress

```

python run_nondeterminism_experiment.py
Fixed input: 'Line 22 - Bus stalled on Main St'
Running 20 runs at temperature=0.7 ...
[temp=0.7] run 1/20...
[temp=0.7] run 2/20...
[temp=0.7] run 3/20...
[temp=0.7] run 4/20...
[temp=0.7] run 5/20...
[temp=0.7] run 6/20...
[temp=0.7] run 7/20...
[temp=0.7] run 8/20...
[temp=0.7] run 9/20...
[temp=0.7] run 10/20...
[temp=0.7] run 11/20...
[temp=0.7] run 12/20...
[temp=0.7] run 13/20...
[temp=0.7] run 14/20...
[temp=0.7] run 15/20...
[temp=0.7] run 16/20...
[temp=0.7] run 17/20...
[temp=0.7] run 18/20...
[temp=0.7] run 19/20...
[temp=0.7] run 20/20...
Running 20 runs at temperature=0.0 ...
[temp=0.0] run 1/20...
[temp=0.0] run 2/20...
[temp=0.0] run 3/20...
[temp=0.0] run 4/20...
[temp=0.0] run 5/20...
[temp=0.0] run 6/20...
[temp=0.0] run 7/20...
[temp=0.0] run 8/20...
[temp=0.0] run 9/20...
[temp=0.0] run 10/20...
[temp=0.0] run 11/20...
[temp=0.0] run 12/20...
[temp=0.0] run 13/20...
[temp=0.0] run 14/20...
[temp=0.0] run 15/20...
[temp=0.0] run 16/20...
[temp=0.0] run 17/20...
[temp=0.0] run 18/20...
[temp=0.0] run 19/20...
[temp=0.0] run 20/20...

Saved raw JSON to /Users/himja/Documents/data260-1346/reports/hw01/raw/nondeterminism_raw.json
Saved raw CSV to /Users/himja/Documents/data260-1346/reports/hw01/raw/nondeterminism_raw.csv

=== TEMPERATURE 0.7 SUMMARY ===
{
  "n_runs": 20,
  "distinct_tag_sets": 15,
  "tags_in_all_runs": [],
  "tags_in_exactly_one_run": [
    "Bus",
    "BusBreakdown",
    "DelayedService",
    "Intersection Troubles",
    "MainStIntersection",
    "Route22",
    "Stalled",
    "Stuck",
    "Traffic Congestion"
  ],
  "latency_p50_ms": 12070.6,
  "latency_p95_ms": 14927.9,
  "latency_p99_ms": 16754.7
}

=== TEMPERATURE 0.0 SUMMARY ===
{
  "n_runs": 20,
  "distinct_tag_sets": 1,
  "tags_in_all_runs": [
    "Bus Delays",
    "Intersection",
    "Mechanical Issue"
  ],
  "tags_in_exactly_one_run": [],
  "latency_p50_ms": 11999.4,
  "latency_p95_ms": 13257.0,
  "latency_p99_ms": 13921.5
}

Saved summary to /Users/himja/Documents/data260-1346/reports/hw01/raw/nondeterminism_summary.json
(venv) himja@Himjas-MacBook-Pro code %

```

Fig 28 Full 40 run output and imputed summary statistics

Results Table

From MATRICS.md

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	15	1
Tags in all 20 runs	none	Bus delays, intersection, mechanical issue
Latency p50 (ms)	12070.6	11999.4
Latency p95(ms)	14972.9	13257.0
Latency p99(ms)	16745.7	13921.5
Tags in exactly 1 run	Bus, BusBreakdown, DelayedService, Intersection Troubles, MainStIntersection, Route22, Stalled, Stuck, Traffic Congestion (9 tags)	none

Analysis:

At temperature 0.7, two users submitting the identical incident report would very likely see different tags: 15 of 20 runs produced a unique tag combination, and no single tag appeared in all 20 runs. At temperature 0.0, both users would see the exact same 3 tags and summary every time all 20 runs converged on one tag set. Latency was comparable across temperatures (~12s median for both), so determinism at temp 0.0 cost essentially nothing in latency here.

- **Where variation is acceptable:** a low-stakes internal dashboard suggesting alternative tag phrasings for a human editor to pick from creative variety can be a feature.
- **Where variation is NOT acceptable:** automated regulatory/compliance categorization of transit incidents reported to a transit authority, where consistent, auditable labels are required, temperature 0.0 should be used here.

Part 6

Model Client and Token Accounting

src/model_client.py implements a ModelClient with a single complete(messages, tools=None) interface that every model call in the project goes through, tracking per-turn and cumulative input/output/total token counts. code/hw1_client.py is a CLI demo built on top of it, supporting a /stats command and printing a final session summary on /exit.

```

Everything up-to-date
(himja@Himjas-MacBook-Pro data260-1346 % ls src/
model_client.py
(himja@Himjas-MacBook-Pro data260-1346 % touch code/hw1_client.py
(himja@Himjas-MacBook-Pro data260-1346 % pip install requests
Requirement already satisfied: requests in ./venv/lib/python3.11/site-packages (2.34.2)
Requirement already satisfied: charset_normalizer<4,>=2 in ./venv/lib/python3.11/site-packages (from requests) (3.5.1)
Requirement already satisfied: idna<4,>=2.5 in ./venv/lib/python3.11/site-packages (from requests) (3.19)
Requirement already satisfied: urllib3<3,>=1.26 in ./venv/lib/python3.11/site-packages (from requests) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in ./venv/lib/python3.11/site-packages (from requests) (2026.7.22)
(himja@Himjas-MacBook-Pro data260-1346 % cd code
python hw1_client.py
HW1 CLI client (type /stats for usage stats, /exit to quit)

You: What are common causes of bus breakdowns?

Assistant: Common causes of bus breakdowns include mechanical failures (such as engine issues, transmission problems, or brakes), electrical malfunctions, tire troubles, and fuel issues. Weather conditions can also contribute to breakdowns, particularly heavy rain, snow, or extreme heat.

[turn tokens - input: 39, output: 52, total: 91]

You:

```

Fig 29 Turn 1 questions and response with per-turn counts

```

You: What should a driver do immediately after a breakdown?

Assistant: After a breakdown, a driver should:

1. Turn on the hazard lights to alert other drivers.
2. Park the bus in a safe location away from traffic and obstacles.
3. Ensure the engine is turned off to prevent overheating or fire.
4. Use the emergency brake to secure the vehicle.
5. Move passengers to a safe area if possible and notify them of the situation.
6. Contact the maintenance team or relevant authorities to report the breakdown.
7. Do not leave the scene without permission, as it may be dangerous for others.

[turn tokens - input: 110, output: 111, total: 221]

You: How long should passengers typically wait for a replacement vehicle?

Assistant: The waiting time for a replacement vehicle after a bus breakdown can vary depending on the transit authority's policies and the severity of the breakdown. Typically, passengers are advised to wait for a reasonable amount of time, often around 15-30 minutes, before seeking alternative transportation. However, specific waiting times should be communicated by the transit authority or driver upon the breakdown.

[turn tokens - input: 241, output: 75, total: 316]

You:

```

Fig 30 Turn 2 and 3

```

You: How long should passengers typically wait for a replacement vehicle?

Assistant: The waiting time for a replacement vehicle after a bus breakdown can vary depending on the transit authority's policies and the severity of the breakdown. Typically, passengers are advised to wait for a reasonable amount of time, often around 15-30 minutes, before seeking alternative transportation. However, specific waiting times should be communicated by the transit authority or driver upon the breakdown.

[turn tokens - input: 241, output: 75, total: 316]

You: /stats

--- /stats ---
Turn count: 3
Cumulative input tokens: 390
Cumulative output tokens: 238
Cumulative total tokens: 628
Serialized conversation-history length: 1722 characters

You:

```

Fig 31 stats checkpoint after turn 3

```

You: Should the bus route be temporarily suspended during a breakdown?
Assistant: Whether a bus route should be temporarily suspended during a breakdown depends on the severity of the issue and the transit authority's policies. In many cases, buses are temporarily rerouted or held at a central location to avoid gridlock and minimize delays. A full route suspension is usually only considered if the breakdown is severe and widespread, causing significant disruptions.

[turn tokens - input: 336, output: 69, total: 405]

You: What information should be logged for the incident report?
Assistant: For an incident report, key information to log includes:

- **Date and Time of the Incident**
- **Location**
- **Description of the Incident (e.g., mechanical failure, electrical problem)**
- **Severity of the Incident**
- **Impact on Service (e.g., route disruption, delay)**
- **Contact Information of the Driver and Supervisor**
- **Names and Contact Information of Witnesses**
- **Actions Taken (e.g., rerouting, suspending service)**
- **Timeline of Events**
- **Photos or Videos (if available)**

This information helps in effective incident management and improvement.

[turn tokens - input: 424, output: 125, total: 549]

You: /stats
--- /stats ---
Turn count: 5
Cumulative input tokens: 1150
Cumulative output tokens: 432
Cumulative total tokens: 1582
Serialized conversation-history length: 2962 characters

You: /exit
=== Session summary ===
Total turns: 5
Cumulative input tokens: 1150
Cumulative output tokens: 432
Cumulative total tokens: 1582
(venv) himja@Himjas-MacBook-Pro code %

```

Fig 32 Turn 4 and 5 and stats checkpoint and after exit it will also show you session summary

AGENT.md Verification

AGENT.md states that the model must answer code review requests in a bulleted list with no text. Initially, the model was given the code review request report with just the file name thrown in the prompt and got a lengthy response. When the restriction in the prompt was changed to explicitly state "bulleted list" (shown above), the model learned how to respond appropriately and give a single bullet.

```
(venv) himja@Himjas-MacBook-Pro code % python hw1_client.py
Assistant: The provided function 'add(a, b)' is working correctly and concise. It takes two parameters 'a' and 'b', adds them together, and returns the result. However, there is a minor improvement to be made for readability and Pythonic style: using the '+' operator directly on the parameters ('a + b') is already the most Pythonic way to add two numbers. Here is a slightly refined version:

```python
def add(a, b):
 return a + b
```

This version omits the temporary variable 'result'. Both versions are correct and functional, but the refined one is more concise.
[turn tokens - input: 58, output: 129, total: 187]

You: Following the code review instructions in AGENT.md, review this function: def add(a, b): result = a+b; return(result)

Assistant: The function 'add(a, b)' provided is almost correct but can be made more concise and Pythonic. Here is a revised version:

```python
def add(a, b):
 return a + b
```

This version eliminates the unnecessary 'result' variable.
[turn tokens - input: 224, output: 56, total: 280]

You: /exit
python hw1_client.py
=== Session summary ===
Total turns: 2
Cumulative input tokens: 282
Cumulative output tokens: 185
Cumulative total tokens: 467
(venv) himja@Himjas-MacBook-Pro code % python hw1_client.py
HW1 CLI client (type /stats for usage stats, /exit to quit)

You: When reviewing code, respond using ONLY a bulleted list. Do not use any prose paragraphs, introductions, or conclusions. Each bullet should identify one specific issue, reference the exact line/function, and be one sentence. Do not restate what the code does. Now review this function: def add(a, b): result = a+b; return(result)

Assistant: - Missing colon at the end of the function definition
[turn tokens - input: 102, output: 11, total: 113]

You: /exit
=== Session summary ===
Total turns: 1
Cumulative input tokens: 102
Cumulative output tokens: 11
Cumulative total tokens: 113
```

Fig 33 Bullet only code review verified against the model's output

Conceptual Questions

1. Why is prior conversation context resent with every turn?
Resending the context for every turn is due to Ollama's LLM APIs being stateless. Because there is no memory, the model treats each request as a new independent case. Since the model is not capable of remembering the earlier turns, you have to request the entire chat history and include it in the messages list in every request in order for the model to remember earlier turns. This is why hw1_client.py sends the entire growing list of the history on the next request.
2. What does 'system prompt' mean in relation to a user message?
A system prompt is a piece of context or guidance that is meant to be persistent and helps the user understand the model's behavior for the entire conversation. It is sent at the beginning of the conversation, typically as the first message, and the model ignores it and treats it as a piece of guidance rather than a prompt for a response. A user message is a single turn of input that the model is expected to respond to. In the example, the system prompt was the same for all the five turns, and the user messages were the questions for each of the five turns, respectively.
3. Why do input tokens grow over a conversation?

It is because the history of every user message and every assistant response is resent as input for each turn. If we look at our indexed data, we can see that cumulative input tokens were 390 (after turn 3) and 1150 (after turn 5) nearly tripling in two turns. Each subsequent request resends everything previously said rather than just the last message.

4. What will ultimately slow the model's growth?

There is a fixed, maximum number of tokens (inputs and outputs combined) that can be processed in a single request for a model's context window. As the user's cumulative history reaches this maximum, the model discards, truncates, or summarizes older turns. The request itself also fails. qwen2.5:3b's context window is small, so if a production conversation continued indefinitely (which this demo does not support), it would need to employ sliding windows, summarization, or some other memory management to remain under the bound set, which this demo can not support.

AI use and reflection:

1. What AI used in

- Assisted in generating ideas and providing guidance (for instance, the visual concept of transit signage, the starting structure of the Dockerfile)
- Assisted in solving issues regarding the failure of the ECS deployment and enriched contextual knowledge concerning the Fargate message
- Checked the code I prepared

2. What was done without using AI

- Created the majority of the code in HTML, JS, and Python without the AI having the primary role of a developer in the creation process
- Executed everything by myself in the terminal
- Made all decisions concerning AWS on my own — cluster/service, security group policies, changes of the keys used
- Carried out the conversation and the verification
- Decided on the tags in Part 2

3. A wrong AI output, and how I caught it

- An inaccurate output from the AI, and the method used in identifying the error
- The Dockerfile for the AI executed successfully on the local machine; however, the ECS task aborted with exit code 255, which I found out through CloudWatch logs despite not consulting the AI
- The finalize() method suggested by the AI relied on tags[:3], assuming that Reviewer would return at least 3 tags, although the test run produced only 2 unusual tags

4. What I changed and why it works now

- The image was built again with `docker buildx build --platform linux/amd64`. The task was successfully confirmed to reach the Running state.
- I created my own finalize() so I can patch up the missing tags based on those provided by the Planner instead of just bukly cutting.

Takeaway

- Creating the main logic on my own means I could truly comprehend the code and identify issues with the AI
- AI is good at producing scaffolding and debugging but is not to be relied on for platform-specific issues (such as CPU architecture) or edge-case logic – it requires actual runs
- The non-determinism present at temperature 0.7 made me realize that you cannot actually know if something works by visually checking its operation with only one run there is a need for conducting 40 runs.

Summary:

This assignment provided a working example of a Municipal Transit Incident reporting system consisting of six parts: an HTML/JS form for transit signage that features client-side validation; the use of JSON; and a report on submissions (Parts 1–2); a system running under Docker that has been deployed to AWS ECS Fargate and can be accessed publicly (Part 3); a three-agent pipeline that consists of a Planner, a Reviewer and a Finalizer using a locally executed qwen2.5:3b model; and a model client designed to track tokens and containing a CLI conversation consisting of 5 turns (Part 6). The tests run to determine the results of the system can provide enough evidence to establish the effectiveness of using temperature 0.0 in situations requiring consistent and traceable outcomes; the process of forming the agents engaged in the operation enables to demonstrate the importance of testing code recommended by AI instead of using it only for the reason it was offered by the AI. The code, configuration files and required documents (METRICS.md, AI_USE.md, RUN_LOG.txt, verification.json) have been checked into git and saved in the repository.

Repository:

GitHub: <https://github.com/Himja18/data260-1346>

Tagged hw1 at commit d1c550948d7ebaa9468baf36a9688ae52c3eccbd

Deployed application: <http://18.224.137.47:8446/>